Chuleta de comandos Linux para Sistemas

Nov 3, 2026 · @otrup

Cómo usar esta chuleta

Resume los comandos de los ejercicios 1 a 5 (Debian 13), agrupados por familia. Lo importante no es memorizarlos: es saber **qué quieres conseguir** y **cómo comprobarlo**.

Método para cualquier tarea:

1. **Define el objetivo** en una frase, con palabras en inglés.
2. **Encuentra la herramienta:** Tab Tab, `apropos` o internet.
3. **Aprende a usarla:** `tldr` → `--help` → `man`.
4. **Pruébalo con cuidado:** copia de seguridad del fichero y **validar antes de aplicar**.
5. **Verifica el resultado:** también lo que **sí** debe funcionar, no solo lo que debe fallar.

Regla de oro: antes de pulsar Intro, mira el **prompt** (¿en qué máquina y con qué usuario?) y **lee el comando** (nombres y rutas).

Prompt	Dónde estás
`C:\Users\si>`	Tu PC, terminal `cmd`
`PS C:\Users\si>`	Tu PC, PowerShell
`user@prueba2:~$`	La VM, usuario normal
`root@prueba2:~#`	La VM, como **root**
`MariaDB [tienda]>`	Dentro de MariaDB: solo SQL, acabado en `;`

Ayuda y búsqueda

Comando	Para qué
`apropos palabra` (o `man -k`)	Buscar **qué comando** hace algo, por su descripción
`apropos -a add user`	Que aparezcan **todas** las palabras
`comando --help`	Resumen rápido de opciones

Comando	Para qué
`man comando`	Manual completo
`man 5 passwd`	Una **sección** concreta del manual
`tldr comando`	Ejemplos prácticos (instalar `tldr`)
`which comando`	**Dónde** está un programa (ruta completa)
Tab / Tab Tab	Autocompletar comandos, ficheros, usuarios, tablas
`history` / Ctrl+R	Ver y buscar comandos anteriores

Secciones del manual: (1) comandos de usuario · **(5)** ficheros de configuración (viven en `/etc`) · **(8)** administración (necesitan `sudo`, están en `/usr/sbin`).

Dentro de `man` y `less`: Espacio / b (bajar / subir página) · `/palabra` (buscar) · `n` / `N` (siguiente / anterior) · `g` / `G` (principio / final) · `q` (salir). Para ir a la definición de una opción: `/^ *Opcion`.

Leer el SYNOPSIS: `[ ]` = opcional · sin corchetes = **obligatorio** · `...` = se puede repetir · una opción con un nombre detrás (`-d HOME_DIR`) **lleva valor**; sin nada (`-m`), **no**.

Buscar en internet: añade *debian*, busca en inglés, mira la **fecha**, lee los **comentarios** y otras respuestas, y adapta los ejemplos a tu máquina (nombres de interfaz, usuarios, rutas). Fuentes buenas: manpages.debian.org, ArchWiki, DigitalOcean, Unix & Linux Stack Exchange, Server Fault.

Ficheros, carpetas y navegación

Comando	Para qué
`pwd`	En qué carpeta estoy
`cd /ruta` · `cd carpeta` · `cd ..` · `cd`	Ruta **absoluta** (empieza por `/`) · **relativa** · subir · ir a tu carpeta personal
`ls -l` · `ls -la` · `ls -ld carpeta`	Listar con detalles · incluidos ocultos · la **propia carpeta**, no su contenido

Comando	Para qué
`mkdir -p /a/b` · `rmdir carpeta`	Crear carpetas (y las padres) · borrar una carpeta **vacía**
`cp origen destino` · `mv origen destino`	Copiar · mover o **renombrar**
`rm fichero` · `rm -r carpeta`	Borrar · borrar recursivo (sin vuelta atrás)
`ln -s original enlace`	Crear un **enlace simbólico** (acceso directo)
`cat` · `less` · `head -n 20` · `tail -n 20` · `tail -f`	Ver un fichero entero · paginado · primeras / últimas líneas · en directo
`nano fichero`	Editar (Ctrl+O guardar, Ctrl+X salir, Ctrl+K cortar línea)
`grep texto fichero` · `grep -r texto /carpeta` · `grep -i`	Buscar texto · en toda una carpeta · sin distinguir mayúsculas
`find /ruta -name "*.log" -mtime +7`	Buscar ficheros por nombre y antigüedad
`cut -d: -f1 /etc/passwd`	Sacar una columna de un fichero separado por `:`
`> fichero`	Vaciar un fichero
`zcat fichero.gz`	Ver un fichero comprimido sin descomprimirlo
`uptime -s` · `whoami` · `id`	Hora de arranque · quién soy · mis grupos

Redirecciones y tuberías:

Símbolo	Significado
`>`	Guardar la salida en un fichero (**sobrescribe**)
`>>`	**Añadir** la salida al final
`<`	Leer la entrada desde un fichero (`mariadb tienda < tienda.sql`)
`2>&1`	Mandar también los **errores** al mismo sitio
`|`	Pasar la salida de un comando a otro (`ps aux | grep nginx`)

Trampa de `sudo`: en `sudo comando > fichero`, la redirección la hace **tu usuario**, no root. Con una tubería, cada lado necesita su propio `sudo`: `sudo zcat copia.gz | sudo mariadb tienda`.

Paquetes con apt

Comando	Para qué
`sudo apt update`	Actualizar la **lista** de paquetes disponibles
`sudo apt install paquete`	Instalar
`sudo apt remove paquete`	Desinstalar
`sudo apt upgrade`	Actualizar lo instalado
`apt search palabra`	Buscar cómo se llama un paquete

Repositorios en `/etc/apt/sources.list.d/debian.sources` (Debian 13 = **trixie**). En Debian: `mariadb-server` (no `mysql-server`), `openssh-server`, `nginx`. Para Python, usar un **entorno virtual** (`python3 -m venv venv`), porque `pip install` a nivel de sistema está bloqueado.

Usuarios, grupos y contraseñas

Los comandos **genéricos** (`useradd` …) funcionan en cualquier distribución; `adduser` / `deluser` son de Debian y más interactivos.

Comando	Para qué
`useradd -m -s /bin/bash -G grupo nombre`	Crear usuario con carpeta (`-m`), shell bash y grupo **adicional**
`useradd -m -d /ruta -s /bin/bash nombre`	Con carpeta en una ruta concreta (`-d` + `-m`)
`useradd -r -M -s /usr/sbin/nologin nombre`	Usuario **de sistema** para un servicio, sin carpeta ni sesión
`usermod -aG grupo nombre`	**Añadir** a un grupo (sin `-a` , **sustituye** todos sus grupos)
`usermod -s shell` · `-d ruta -m` · `-l nuevo viejo`	Cambiar shell · mover carpeta · renombrar
`userdel nombre` · `userdel -r nombre`	Borrar · borrar con su carpeta (**mira antes su ruta**)
`groupadd` · `groupmod -n nuevo viejo` · `groupdel`	Crear · renombrar · borrar grupo
`gpasswd -a usuario grupo` · `gpasswd -d usuario grupo`	Añadir · quitar de un grupo
`passwd nombre`	Poner o cambiar la contraseña
`passwd -e nombre` (o `chage -d 0`)	Obligar a cambiarla en el próximo inicio de sesión
`passwd -S nombre`	Estado: **P** = tiene contraseña, **L** = bloqueada
`chage -l nombre` · `chage -E -1 nombre`	Ver caducidad · quitar caducidad de la cuenta
`id nombre` · `getent passwd nombre` · `getent group grupo`	Comprobar usuario, grupos y miembros
`su - nombre` · `exit`	Entrar como otro usuario (con **guion**) · volver

`-g` frente a `-G`: `-g` = grupo **principal** (uno solo; el de los ficheros que crea) · `-G` = grupos **adicionales** (varios).

Orden de las contraseñas: primero `passwd usuario`, después `passwd -e usuario`. Poner una contraseña nueva **borra** la caducidad.

Ficheros: `/etc/passwd` = `usuario:x:UID:GID:info:carpeta:shell` · `/etc/group` = `grupo:x:GID:miembros` · `/etc/shadow` = contraseñas cifradas y caducidad. Valores por defecto: `useradd -D`. Usuarios humanos desde el UID **1000**; de sistema, por debajo.

`getent`: consultar usuarios, grupos y nombres

`getent` (*get entries*) pide al sistema **una entrada** de una de sus "bases de datos", **por el mismo camino** que usan todos los programas. Por eso es mejor que leer los ficheros a mano: muestra solo lo que pides y lo que el sistema **ve de verdad**.

Comando	Para qué
`getent passwd usuario`	La línea de un usuario (UID, carpeta, shell)
`getent group grupo`	Un grupo y sus miembros
`getent hosts nombre`	A qué IP resuelve un nombre (usa `/etc/hosts` y el DNS)
`getent ahosts nombre` · `getent ahostsv4 nombre`	Todas las direcciones · solo IPv4
`getent services ssh`	Qué puerto corresponde a un servicio (22)

El **orden** en que busca cada cosa está en `/etc/nsswitch.conf`. Para nombres: `hosts: files myhostname dns` → primero `/etc/hosts`, después el nombre de la propia máquina y, por último, el DNS. Si un nombre no resuelve como esperas, mira ese orden y `/etc/hosts`.

Cómo comprobarlo: usuarios y grupos

Qué compruebas	Comando	Qué debe salir
Grupos de un usuario	`id ana`	`gid=` su grupo principal y, en `groups=`, los adicionales
Carpeta y shell	`getent passwd ana`	`/home/ana` y `/bin/bash` (servicio: `/nonexistent` y `/usr/sbin/nologin`)

Qué compruebas	Comando	Qué debe salir
Que la carpeta existe	`ls -ld /home/ana`	`drwx------ ana ana`
Miembros de un grupo	`getent group desarrollo`	`desarrollo:x:GID:ana,luis`
Que tiene contraseña	`passwd -S ana`	`P` (`L` = bloqueada, sin contraseña)
Cambio obligatorio al entrar	`chage -l ana`	*password must be changed*
Política de caducidad	`chage -l marta`	*Maximum* 90, *warning* 7
Caducidad de la cuenta	`chage -l becario`	*Account expires*: la fecha
Que no puede iniciar sesión	`su - monitor`	*This account is currently not available*
Prueba real	`su - ana` → `whoami` → `exit`	Pide cambiar la contraseña; prompt `ana@...`

Permisos y propietarios

`chown` decide **de quién** es; **`chmod`**, **qué puede hacer** cada uno. Lo normal: primero `chown`, después `chmod`.

Comando	Para qué
`chown usuario:grupo ruta`	Cambiar propietario y grupo
`chown :grupo ruta` · `chown usuario: ruta`	Solo el grupo · propietario + su grupo principal
`chown -R usuario:grupo carpeta`	También todo lo de dentro (cuidado con la ruta)
`chmod 770 ruta`	Permisos en **octal** (sustituye todos)
`chmod g+s ruta` · `chmod +t ruta` · `chmod +x script`	Permisos en **simbólico** (añade o quita uno)

Comando	Para qué
`umask 077`	Lo que se cree a partir de ahí, solo para el propietario

Leer `drwxrws--T root desarrollo`: tipo (`d` carpeta, `-` fichero, `l` enlace) · propietario `rwx` · grupo `rws` · otros `--T` · propietario y grupo.

Número	Permiso	En un fichero	En una carpeta
4	`r`	Leer	**Listar** el contenido
2	`w`	Modificar	Crear y **borrar** dentro
1	`x`	Ejecutar	**Entrar** (`cd`)

Bits especiales (el cuarto número, delante: `3770`):

Valor	Bit	En una carpeta	Se ve como
4	setuid	(en ficheros: se ejecuta como su dueño)	`s` en propietario
2	**setgid**	Lo nuevo **hereda el grupo** de la carpeta	`s` / `S` en grupo
1	**sticky**	Solo el dueño de un fichero puede **borrarlo** (como `/tmp`)	`t` / `T` en otros

Minúscula = con `x` debajo; mayúscula = sin `x`.

Valores típicos: `755` programas y carpetas públicas · `644` ficheros normales · `700` / `600` solo el dueño (`~/.ssh` , `authorized_keys` , copias) · `750` script de administración · `0440` reglas de sudoers.

Borrar un fichero depende de los permisos de la **carpeta**; **modificarlo**, de los del **fichero**. En carpetas con sticky, `fs.protected_regular = 2` impide también escribir en ficheros ajenos.

Carpeta compartida donde todos editan lo de todos: setgid + sticky + ACL

La idea clave: la carpeta y el fichero tienen permisos distintos. Piensa en la carpeta como un **armario** y en cada fichero como una **caja** dentro.

Permisos de...	Deciden...	Ejemplo (`historiales` , `3770 root:clinica`)
La **carpeta**	Quién puede **meter, sacar o renombrar cajas** en el armario	Grupo `rwx` : los dos pueden crear ficheros
Cada **fichero**	Quién puede **escribir dentro de esa caja**	Lo decide la **umask** de quien la crea

«El grupo tiene `rwx` » es verdad **para la carpeta**. Editar `h1.txt` no es tocar la carpeta: es tocar el fichero, que tiene sus propios permisos.

Qué hace cada número del `3770` . El `3` son **dos bits juntos** (2 + 1), y **ninguno da permisos de escritura**:

Bit	Valor	Qué hace	Qué NO hace
setgid	2	Pone la etiqueta de grupo (`clinica`) a cada caja nueva	No le da `w` al grupo
sticky	1	Solo el dueño de una caja puede sacarla del armario (borrar o renombrar)	No impide ni permite editar
`770`	—	Permisos **de la carpeta**: dueño y grupo entran y crean	No se copian a los ficheros

En carpetas, el **setuid** no hace nada en Linux.

Lo que pasa SIN ACL, paso a paso (`lgarcia` crea `h1.txt`):

1. Los programas crean los ficheros pidiendo `rw-rw-rw-` (666).
2. La **umask** de `lgarcia` (`022`) le quita la `w` al grupo y a otros, y queda `rw-r--r--` .
3. El setgid pone el grupo `clinica` .

Resultado: `-rw-r--r-- lgarcia clinica h1.txt` . `pmarin` está en `clinica` , pero el grupo solo tiene `r--` : **lee, no escribe**. Nada del `3770` lo cambia.

La ACL por defecto = una norma del armario: «toda caja nueva nace con escritura para el grupo». Gana a la umask. Con ella, `h1.txt` nace `-rw-rw---- lgarcia clinica` y `pmarin` sí puede editarlo. El `+` indica que hay ACL: sale en la carpeta (ls -ld), no en el fichero nuevo, que solo hereda permisos normales.

El comando, pieza a pieza:

```
setfacl -d -m g::rwx /srv/clinica/historiales
```

Pieza	Significa
`setfacl`	*set file ACL*: poner una ACL
`-m`	*modify*: añadir o cambiar una entrada
`-d`	*default*: la entrada es para **lo que se cree dentro**, no para la carpeta misma (sin `-d` solo cambia la carpeta)
`g::rwx`	La entrada: `tipo:quién:permisos`
`/srv/...`	La carpeta

La entrada `tipo:quién:permisos`:

Parte	Opciones	En `g::rwx`
tipo	`u` usuario · `g` grupo · `o` otros	`g` = grupo
quién	**Vacío** = el propietario (dueño o grupo de la carpeta) · con nombre = uno concreto	Vacío = el grupo de la carpeta (`clinica`)
permisos	`r`, `w`, `x` o `-`	`rwx`

Por eso hay **dos puntos seguidos**: la parte «quién» está vacía.

Entrada	Significa
`g::rwx`	El grupo propietario de la carpeta: rwx
`g:clinica:rwx`	El grupo `clinica` en concreto: rwx
`u:pmarin:r--`	El usuario `pmarin`: solo lectura
`o::---`	Otros: nada

Los ficheros nuevos salen con `rw-` aunque pongas `rwx`: los programas nunca crean ficheros con `x`. Las **subcarpetas** nuevas sí reciben la `x`, que necesitan para entrar.

Cómo leer `getfacl`:

```
# flags: -st          ← setgid (s) y sticky (t)
user::rwx            ← permisos de la CARPETA
group::rwx
other::---
default:user::rwx    ← la NORMA para lo que se cree dentro
default:group::rwx   ← los ficheros nuevos nacen con el grupo en rw
default:other::---
```

Sin `default:` = la carpeta. Con `default:` = la plantilla de lo nuevo.

En una frase: el setgid decide **de qué grupo** es el fichero nuevo, el sticky **quién puede borrarlo** y la ACL por defecto **qué puede hacer el grupo** con él.

Ejemplo completo (`/srv/clinica/historiales` , grupo `clinica` , usuarios `lgarcia` y `pmarin`):

```
apt install acl                                    # solo si setfacl da
"command not found"
mkdir -p /srv/clinica/historiales
chown root:clinica /srv/clinica/historiales
chmod 3770 /srv/clinica/historiales                # setgid + sticky + rwx
dueño y grupo
setfacl -d -m g::rwx /srv/clinica/historiales      # lo nuevo nace con el
grupo en rw
setfacl -d -m o::--- /srv/clinica/historiales      # lo nuevo nace sin nada
para otros
chmod g+rw /srv/clinica/historiales/*              # solo si ya había
ficheros (la ACL por defecto no los toca)
getfacl /srv/clinica/historiales                   # deben salir
default:group::rwx y default:other::---
```

Quitar la ACL por defecto si te equivocas: `setfacl -k carpeta` . Quitar todas: `setfacl -b carpeta` .

Pruebas (como los usuarios, nunca como root):

Comando	Resultado esperado	Qué demuestra
`su - lgarcia -c 'echo historial > /srv/clinica/historiales/h1.txt'`	Funciona	`lgarcia` crea
`ls -l /srv/clinica/historiales/h1.txt`	`-rw-rw---- lgarcia clinica`	Grupo `clinica` y `rw` del grupo

Comando	Resultado esperado	Qué demuestra
`su - pmarin -c 'echo nota >> /srv/clinica/historiales/h1.txt'`	Funciona (sin ACL: `Permission denied`)	`pmarin` edita lo de `lgarcia`
`su - pmarin -c 'rm /srv/clinica/historiales/h1.txt'`	`Operation not permitted`	El sticky protege el fichero
`su - user -c 'ls /srv/clinica/historiales'`	`Permission denied`	El resto no entra

Alternativa sin ACL: `umask 002` en cada usuario. Es frágil: depende de cada sesión y de cada programa. La ACL va en la carpeta y vale para todos.

ACL para un usuario concreto que no está en el grupo

Caso: `/srv/taller/recambios`, el grupo `recambios` lee y escribe, y `sfuentes` (que **no** está en el grupo) **solo lee**.

No lo hagas dueño para limitarlo: el dueño siempre puede cambiar los permisos de lo suyo (`chmod 770` y ya escribe). A alguien de fuera con un permiso concreto se le da con una ACL.

```
chown root:recambios /srv/taller/recambios
chmod 2770 /srv/taller/recambios                        # setgid + rwx dueño y grupo, nada otros
setfacl -m u:sfuentes:rx /srv/taller/recambios          # la CARPETA: listar (r) y entrar (x), sin crear (sin w)
setfacl -d -m u:sfuentes:rx /srv/taller/recambios       # lo NUEVO: los ficheros le llegan con r
getfacl /srv/taller/recambios
```

Pieza	Significa
`-m` sin `-d`	ACL de la carpeta misma
`-d -m`	ACL por defecto: plantilla para lo que se cree dentro
`u:sfuentes:rx`	Tipo usuario · quién: `sfuentes` · permisos: `r-x`
En una carpeta: `r`	Ver qué hay dentro (`ls`)

Pieza	Significa
En una carpeta: `x`	Entrar y abrir lo de dentro. Sin `x`, `r` sola no deja leer los ficheros
En una carpeta: `w`	Crear, borrar y renombrar dentro

`getfacl` debe mostrar `user:sfuentes:r-x`, una línea **`mask::rwx`** y `default:user:sfuentes:r-x`. La `mask` sale sola al poner una ACL de usuario: es el **máximo** que pueden tener esas entradas (si fuera `r--`, `sfuentes` no podría entrar aunque su línea diga `r-x`).

Pruebas:

Comando	Tiene que salir
`su - rortiz -c 'echo pieza > /srv/taller/recambios/r1.txt'`	Funciona
`su - sfuentes -c 'ls /srv/taller/recambios; cat /srv/taller/recambios/r1.txt'`	`r1.txt` y `pieza`
`su - sfuentes -c 'touch /srv/taller/recambios/x'`	`Permission denied`
`su - sfuentes -c 'chmod 777 /srv/taller/recambios'`	`Operation not permitted`
`su - becario -c 'ls /srv/taller/recambios'`	`Permission denied`

Quitar solo la entrada de `sfuentes`: `setfacl -x u:sfuentes carpeta` (y `setfacl -d -x u:sfuentes carpeta` para la de por defecto).

Cómo comprobarlo: permisos

Qué compruebas	Comando	Qué debe salir
Permisos de la carpeta	`ls -ld /srv/proyecto`	`drwxrws--T root desarrollo` (3770)
Permisos de lo de dentro	`ls -l /srv/proyecto`	Ficheros nuevos con grupo `desarrollo` (setgid)

Qué compruebas	Comando	Qué debe salir
El grupo puede escribir	`su - ana` → `touch /srv/proyecto/f`	Sin error
Nadie borra lo ajeno	`su - luis` → `rm /srv/proyecto/f`	*Operation not permitted* (sticky)
Los demás no entran	`su - soporte` → `ls /srv/proyecto`	*Permission denied*
Probar como otro usuario sin sesión	`sudo -u vigia comando`	Ejecuta como ese usuario

Permission denied = faltan permisos normales · *Operation not permitted* = los tiene, pero el sticky bit lo impide. Mira siempre el **prompt** antes de cada prueba: si eres root, todo funciona y la prueba no demuestra nada.

sudo y sudoers

Comando	Para qué
`sudo comando`	Ejecutar como root, con **tu** contraseña
`sudo -i`	Abrir una shell de root (prompt `#`); salir con `exit`
`sudo -l`	Ver qué puede hacer el usuario con sudo
`visudo -f /etc/sudoers.d/nombre`	Crear o editar una regla de forma segura
`visudo -c`	Validar toda la configuración de sudo
`chmod 0440 /etc/sudoers.d/nombre`	Los permisos que exige sudo

`su` frente a `sudo`: `su` pide la contraseña **del usuario al que cambias** (root no tiene en esta VM, así que `su` a secas no funciona); `sudo` pide **la tuya**. `-i` es de `sudo`, no de `su`.

Formato de una regla: `quién dónde=(como quién) qué comando`

```
soporte  ALL=(root) /usr/bin/systemctl restart ssh
%soporte ALL=(root) /usr/bin/systemctl restart ssh
%sudo    ALL=(ALL:ALL) ALL
```

Varios comandos para un grupo (separados por coma), en `/etc/sudoers.d/operadores`:

```
%operadores ALL=(root) /usr/bin/systemctl status nginx, /usr/bin/systemctl restart nginx
```

Dentro del fichero va la **regla**, sin `sudo` delante. El usuario la usa después en la terminal: `sudo systemctl restart nginx`. Para dar o quitar el permiso a alguien basta con el grupo: `usermod -aG operadores pedro` / `gpasswd -d pedro operadores`.

- `%` delante = un **grupo**.
- Ruta **completa** del comando (`which systemctl`) y sus **argumentos**: sin argumentos, permitirías cualquier uso.
- `(root)` mejor que `(ALL)`; sin `NOPASSWD` salvo para automatizaciones.
- El fichero de `sudoers.d` **sin punto** en el nombre, o se ignora.
- `visudo -c` comprueba la **sintaxis**, no que la regla haga lo que quieres: prueba con `sudo -l` y ejecutando el comando.

Mínimo privilegio: antes de dar sudo, pregúntate si basta con un **grupo** (por ejemplo, `adm` o `systemd-journal` para leer logs).

Cómo comprobarlo: sudo

Qué compruebas	Comando	Qué debe salir
Sintaxis y permisos de las reglas	`visudo -c`	*parsed OK* en todos los ficheros
Permisos del fichero	`ls -l /etc/sudoers.d/soporte`	`-r--r----- root root`
Qué puede hacer el usuario	`su - soporte` → `sudo -l`	Solo su regla, con la ruta completa
Lo permitido funciona	`sudo systemctl restart ssh`	Sin mensaje
Lo demás se deniega	`sudo systemctl restart cron` · `sudo -i`	*Sorry, user … is not allowed*
Otro usuario no tiene sudo	`su - marta` → `sudo -l`	*may not run sudo*

Servicios con systemd y logs

Para servicios, siempre **`systemctl` + acción + servicio**. Mirar no necesita root; cambiar, sí.

Comando	Para qué
`systemctl status ssh --no-pager`	Estado, PID y últimos logs
`systemctl start` · `stop` · `restart` · `reload`	Arrancar · parar · reiniciar · recargar la configuración sin cortar
`systemctl enable` · `disable` · `enable --now`	Arranque automático · quitarlo · habilitar y arrancar ya
`systemctl is-enabled` · `is-active`	Respuesta corta: `enabled` / `active`
`systemctl cat servicio`	Ver el fichero `.service`
`systemctl show servicio -p Restart`	Ver el valor que systemd **tiene cargado**
`systemctl daemon-reload`	Releer los `.service` tras crearlos o editarlos
`systemctl list-units --type=service`	Ver qué servicios hay
`systemctl get-default` · `set-default multi-user.target`	Arranque en modo gráfico o texto
`journalctl -u servicio -n 50` · `-f` · `-b` · `--since "19:00"`	Logs de un servicio · en directo · este arranque · desde una hora
`journalctl -xe`	Errores recientes explicados

Servicio propio en `/etc/systemd/system/nombre.service` (el nombre del fichero es el del servicio):

```
[Unit]
Description=Monitor de uso de disco
After=local-fs.target

[Service]
Type=simple
User=monitor
Group=monitor
ExecStart=/usr/local/bin/monitor.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Después: `daemon-reload` → `enable --now` → `status`.

`Restart=`: `on-failure` reinicia si falla (`kill -9`), pero no con un `kill` normal; `always` reinicia siempre. `systemctl stop` nunca reinicia.

`enable` crea un enlace en `multi-user.target.wants/` (o `sysinit.target.wants/`, como el firewall, que arranca antes que la red). **`active (exited)`** es correcto en servicios que cargan algo y terminan, como `nftables`.

Crear un servicio paso a paso

Son **dos ficheros distintos**: el **script** dice **qué** hacer; el **`.service`**, **cómo** ejecutarlo. El `.service` no contiene el script: lo **apunta** con `ExecStart`.

1. **El script** en `/usr/local/bin/nombre.sh`, de root, con `chmod 755` (o `750`).
2. **Pruébalo a mano** y después **como el usuario del servicio**: `sudo -u usuario /usr/local/bin/nombre.sh`.
3. **El usuario** del servicio, si no debe ser root: `useradd -r -M -d /nonexistent -s /usr/sbin/nologin usuario`, y `chown -R` + `chmod 750` en su carpeta de trabajo (las carpetas necesitan la `x`).
4. **El servicio** en `/etc/systemd/system/nombre.service` (el nombre del fichero es el del servicio).
5. **Releer:** `systemctl daemon-reload` (sin argumentos: relee todos; **no** ejecuta nada).
6. **Usarlo:** `systemctl start` (ahora) · `enable` (al encender) · `enable --now` (las dos) · o un `.timer` del **mismo nombre** que lo lanza periódicamente.

7. **Comprobar:** `systemctl status nombre`, `journalctl -u nombre` y el resultado real (el log, el puerto…).

Estructura de un `.service`:

```
[Unit]
Description=Qué hace el servicio
After=local-fs.target

[Service]
Type=simple
User=usuario
Group=usuario
ExecStart=/usr/local/bin/nombre.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Sección / opción	Significado
`[Unit]`	Descripción y **dependencias** del servicio
`Description=`	El texto que muestra `systemctl status`
`After=`	Arrancar **después** de algo: `local-fs.target` (discos montados), `network.target` (red)
`[Service]`	**Cómo** se ejecuta
`Type=simple`	El programa **se queda en marcha** (un bucle, un servidor). Estado normal: `active (running)`
`Type=oneshot`	El programa **hace su trabajo y termina**. Estado normal: `inactive (dead)` tras ejecutarse
`User=` · `Group=`	Con qué usuario y grupo se ejecuta (nunca root si no hace falta)
`ExecStart=`	El programa a ejecutar, con **ruta completa**
`Restart=`	`always` (siempre) · `on-failure` (solo si falla). No tiene sentido en `oneshot`

Sección / opción	Significado
`RestartSec=`	Segundos de espera antes de reiniciar
`[Install]`	**Cuándo** arranca si se hace `enable`
`WantedBy=multi-user.target`	Arrancar con el sistema ya iniciado. Un servicio lanzado por un `.timer` no lo necesita: el que se habilita es el temporizador

Errores típicos: editar el `.service` y olvidar `daemon-reload` (`systemctl show nombre -p Opcion` muestra lo cargado) · el fichero de log creado por root al probar a mano (`chown -R`) · una carpeta sin `x` (`660` en vez de `750`) · un nombre de servicio distinto del que espera el `.timer`.

Ejemplo completo: dónde va cada fichero

Un vigilante que comprueba la web cada minuto, con su usuario propio:

```
/usr/local/bin/vigila-web.sh          → el script (QUÉ hace)         root,
755
/etc/systemd/system/vigia.service     → el servicio (CÓMO se ejecuta)
/etc/systemd/system/vigia.timer       → el temporizador (CUÁNDO)
/var/log/vigia/web.log                → su log                       vigia,
carpeta 750
```

Contenido de `/etc/systemd/system/vigia.service`:

```
[Unit]
Description=Vigilante de la web

[Service]
Type=oneshot
User=vigia
Group=vigia
ExecStart=/usr/local/bin/vigila-web.sh
```

Sin `[Install]` ni `Restart=`: es `oneshot` y lo lanza `vigia.timer`. Después: `daemon-reload` → `sudo -u vigia /usr/local/bin/vigila-web.sh` (prueba) → `systemctl enable --now vigia.timer` → `list-timers` y `journalctl -u vigia.service`.

Temporizadores (`.timer`)

Un `.timer` es otro fichero de configuración de systemd que dice **cuándo** lanzar un servicio. Sustituye a cron y a los bucles con `sleep` : el script se ejecuta **una vez** (`Type=oneshot`) y systemd lo vuelve a lanzar según el calendario.

```
vigia.timer    →  CUÁNDO  (cada minuto)
vigia.service  →  CÓMO    (usuario, tipo oneshot)
vigila-web.sh  →  QUÉ     (el trabajo)
```

En `/etc/systemd/system/nombre.timer` :

```
[Unit]
Description=Lanza el vigilante de la web cada minuto

[Timer]
OnCalendar=minutely

[Install]
WantedBy=timers.target
```

Opción	Significado
`OnCalendar=`	Cuándo, como un calendario: `minutely` · `hourly` · `daily` · `*-*-* 02:30:00` (cada día a las 2:30) · `Mon *-*-* 03:00:00` (los lunes)
`OnBootSec=` · `OnUnitActiveSec=`	Tiempo desde el arranque · desde la última ejecución (`5min` , `1h`)
`Persistent=true`	Si la máquina estaba apagada a la hora prevista, lo ejecuta al encender
`Unit=`	Otro servicio a lanzar. Sin esta línea, lanza el **del mismo nombre** (`vigia.timer` → `vigia.service`)
`WantedBy=timers.target`	Que el temporizador arranque al encender la máquina

Pasos: crear el `.timer` → `systemctl daemon-reload` → `systemctl enable --now nombre.timer` (se habilita el **temporizador**, no el servicio) → comprobar.

Comando	Para qué
`systemctl list-timers`	Todos los temporizadores: próxima (`NEXT`) y última (`LAST`) ejecución y qué servicio lanzan
`systemd-analyze calendar "expresión"`	Comprobar una expresión de `OnCalendar` antes de usarla
`journalctl -u nombre.service`	Los logs de cada ejecución

Frente a cron: cada ejecución queda registrada en el journal, se puede comprobar con `list-timers` , usa el usuario y las protecciones del `.service` , y con `Persistent=true` no se pierden ejecuciones si la máquina estaba apagada.

Cómo comprobarlo: servicios y temporizadores

Qué compruebas	Comando	Qué debe salir
Está en marcha	`systemctl status monitor --no-pager`	*active (running)* (`oneshot` : *inactive (dead)* tras ejecutarse)
Arranca al encender	`systemctl is-enabled monitor`	`enabled`
Con qué usuario corre	`ps -o user,pid,cmd -C monitor.sh`	`monitor` , no root
Qué tiene cargado systemd	`systemctl show monitor -p Restart`	`Restart=always`
Que hace su trabajo	`tail -n 5 /var/log/monitor/monitor.log`	Líneas nuevas cada 10 s
Se levanta si falla	`kill -9 PID` → `systemctl status monitor`	PID nuevo; *Scheduled restart job*
Sus logs	`journalctl -u monitor -n 20 --no-pager`	Arranques, paradas, reinicios
Temporizador	`systemctl list-timers vigia.timer`	`NEXT` , `LAST` y `ACTIVATES vigia.service`

Qué compruebas	Comando	Qué debe salir
Sobrevive al reinicio	`reboot` → `status` + `uptime -s`	Activo justo después del arranque

Procesos y señales

Comando	Para qué
`ps`	Solo los procesos de tu terminal
`ps aux`	Todos los procesos del sistema
`ps -o user,pid,cmd -C nombre`	Un proceso concreto y **quién** lo ejecuta
`top` / `htop`	Procesos en directo, por consumo
`kill PID`	Señal **SIGTERM**: terminar de forma ordenada
`kill -9 PID`	Señal **SIGKILL**: matar a la fuerza
`Ctrl+C`	Parar el programa en primer plano
`echo $?`	Código de salida del último comando: `0` = bien

Un programa lanzado dentro de una sesión SSH **se para al cerrar esa sesión**. Para que sobreviva, conviértelo en un **servicio de systemd**.

Scripts de Bash y cron

Esqueleto de un script (en `/usr/local/bin`, con `chmod +x` o `750`):

```
#!/bin/bash
set -euo pipefail
umask 077

FECHA=$(date +%Y-%m-%d_%H-%M-%S)
FICHERO="/var/backups/mariadb/tienda_${FECHA}.sql.gz"

mkdir -p /var/backups/mariadb
mariadb-dump --single-transaction tienda | gzip > "$FICHERO"
gzip -t "$FICHERO"
find /var/backups/mariadb -name 'tienda_*.sql.gz' -mtime +7 -delete
echo "Backup correcto: $FICHERO"
```

Pieza	Para qué
`#!/bin/bash`	Qué intérprete ejecuta el script
`set -e` · `-u` · `-o pipefail`	Parar si algo falla · si una variable no existe · si falla cualquier parte de una tubería
`VAR=valor` · `"$VAR"` · `${VAR}_texto`	Definir · usar (entre comillas) · delimitar el nombre
`$(comando)`	Meter la salida de un comando en una variable o texto
`while true; do ...; sleep 10; done`	Bucle infinito cada 10 segundos
`bash -x script.sh`	**Depurar**: muestra cada línea antes de ejecutarla

cron (`sudo crontab -e` para root · `crontab -l` para ver):

```
# minuto hora día-mes mes día-semana  comando
30 2 * * * /usr/local/bin/backup-tienda.sh >> /var/log/backup-tienda.log 2>&1
```

Expresión	Cuándo
`30 2 * * *`	Cada día a las 2:30
`0 * * * *`	Cada hora en punto
`*/5 * * * *`	Cada 5 minutos

Expresión	Cuándo
`0 3 * * 1`	Los lunes a las 3:00

En cron: **rutas completas**, el `%` se escribe `\%`, y redirige la salida a un **log** con `2>&1`. Prueba el script **a mano** antes de programarlo, y comprueba después que se ha ejecutado.

Cómo comprobarlo: scripts y cron

Qué compruebas	Comando	Qué debe salir
El script funciona	`sudo /usr/local/bin/backup-tienda.sh` → `echo $?`	Su mensaje y `0`
Dónde falla	`sudo bash -x /usr/local/bin/script.sh`	Cada línea con `+`, hasta dónde llega
Resultado y permisos	`sudo ls -l /var/backups/mariadb/`	Ficheros con fecha, `-rw-------`
Contenido de una copia	`sudo zcat copia.sql.gz | head -n 30`	Los `CREATE TABLE`
Tarea programada	`sudo crontab -l`	La línea con la hora y la ruta completa
Que cron la ejecutó	`sudo tail /var/log/backup-tienda.log`	El mensaje de la madrugada

Red

Los cuatro datos: IP (`192.168.1.200`) · máscara (`/24`) · puerta de enlace (`192.168.1.1`) · DNS (`192.168.1.136`).

Comando	Para qué
`ip a` (`ip address`) · `ip a show enp0s3`	Interfaces e IPs (`dynamic` = por DHCP)
`ip r`	Rutas: `default via` = puerta de enlace
`cat /etc/resolv.conf`	Servidores DNS

Comando	Para qué
`ping -c 3 IP` · `ping -4 -c 3 nombre`	¿Llego? · forzando IPv4
`ss -tulpn` (con `sudo`, muestra el programa)	Puertos que escuchan y quién
`curl -I http://host` · `curl -m 5 ...`	Cabeceras HTTP · rendirse a los 5 segundos
`curl -H "Host: nombre" http://localhost`	Simular el nombre que pide un navegador
`curl -m 5 telnet://IP:puerto`	¿Está abierto un puerto?
`systemctl status NetworkManager`	¿Quién gestiona la red?

Diagnóstico de abajo arriba: ¿tengo IP? → ¿llego al router? → ¿llego a `1.1.1.1`? → ¿resuelve el DNS?

IP estática con NetworkManager (VM con escritorio):

```
nmcli connection show
sudo nmcli connection modify "Wired connection 1" ipv4.method manual
ipv4.addresses 192.168.1.200/24 ipv4.gateway 192.168.1.1 ipv4.dns
192.168.1.136
sudo nmcli connection up "Wired connection 1"
```

`nmcli`: la herramienta de NetworkManager

NetworkManager es el servicio que gestiona la red en Debian con escritorio; **`nmcli`** (*NetworkManager Command Line Interface*) es el comando para darle órdenes, igual que `systemctl` con systemd. Trabaja con **conexiones** (perfiles de configuración), no con interfaces: el perfil `Wired connection 1` se aplica a `enp0s3`.

Comando	Para qué
`nmcli device status`	Interfaces y qué conexión usa cada una
`nmcli connection show`	Lista de conexiones (el nombre exacto, entre comillas si tiene espacios)
`nmcli connection show "Wired connection 1"`	Todos sus ajustes
`nmcli -g ipv4.addresses connection show "Wired connection 1"`	Un ajuste concreto

Comando	Para qué
`nmcli connection modify "..." ipv4.addresses 192.168.1.200/24`	**Sustituir** todas las direcciones
`nmcli connection modify "..." +ipv4.addresses 192.168.1.201/24`	**Añadir** una dirección (`-` para quitarla)
`nmcli connection up "..."`	**Aplicar** los cambios (reactivar la conexión)
`nmcli connection reload`	Releer los ficheros si se editaron a mano

Propiedades típicas: `ipv4.method manual` (fija) o `auto` (DHCP) · `ipv4.addresses` · `ipv4.gateway` · `ipv4.dns`.

⚠️ **Sin `+` se sustituyen TODAS las direcciones.** `ipv4.addresses 192.168.1.202/24` deja solo la `.202` y borra la principal (y con ella tu SSH). Para cambiar solo una IP secundaria:

```
ping -c 3 192.168.1.202                                    # comprobar que está libre
nmcli connection modify "Wired connection 1" -ipv4.addresses 192.168.1.201/24
nmcli connection modify "Wired connection 1" +ipv4.addresses 192.168.1.202/24
nmcli connection up "Wired connection 1"
```

O todas de una vez, separadas por comas y con la **principal primero**: `ipv4.addresses "192.168.1.200/24,192.168.1.202/24"`. Si te quedas sin red, se arregla desde la consola igual. El nombre de la conexión distingue mayúsculas: `Wired connection 1`.

`modify` **guarda** el cambio en `/etc/NetworkManager/system-connections/` (por eso es permanente), pero **no lo aplica** hasta `up`. Alternativa con menús: `nmtui`.

Herramienta	Efecto	Tras reiniciar
`ip address add`	Inmediato, en el kernel	Se pierde
`nmcli` (NetworkManager)	Se guarda y se aplica con `up`	Se mantiene
`/etc/network/interfaces` (ifupdown)	Se aplica con `systemctl restart networking`	Se mantiene

No mezclar gestores: si la red la lleva NetworkManager, no tocar `/etc/network/interfaces` para esa interfaz.

IP estática con ifupdown (servidor sin escritorio), en `/etc/network/interfaces` y después `systemctl restart networking`:

```
auto enp0s3
iface enp0s3 inet static
    address 192.168.1.200/24
    gateway 192.168.1.1
    dns-nameservers 192.168.1.136
```

`ip address add` cambia la IP **al momento pero no se guarda**: solo para pruebas.

`bind`: `127.0.0.1:puerto` = solo la propia máquina · `0.0.0.0` / `[::]` = todas las interfaces, accesible desde la red.

Errores: *Connection refused* inmediato = nadie escucha · *timed out* = el firewall descarta (`drop`).

Cómo comprobarlo: red

Qué compruebas	Comando	Qué debe salir
IPs de la interfaz	`ip a show enp0s3`	Las IPs, **sin** `dynamic` si son fijas
Que la IP está guardada	`nmcli -g ipv4.addresses connection show "Wired connection 1"`	Todas las direcciones del perfil
Puerta de enlace	`ip r`	`default via 192.168.1.1`
DNS	`cat /etc/resolv.conf`	`nameserver 192.168.1.136`
Conectividad, por capas	`ping -c 3 192.168.1.1` → `ping -4 -c 3 1.1.1.1` → `getent hosts debian.org`	Router → internet → DNS
Nombre de la máquina	`hostnamectl` · `getent ahostsv4 srv-tienda`	El nombre · `127.0.1.1`
Qué está expuesto	`sudo ss -tlpn`	Solo 22 y 80 en `0.0.0.0` / `[::]`; lo demás en `127.0.0.1`

Qué compruebas	Comando	Qué debe salir
Accesible desde fuera	Desde el PC: `ping` a cada IP	Respuesta (el `time` real, no 0,04 ms)
Se mantiene	`reboot` → repetir	Igual que antes

SSH

`ssh` = el cliente (te conectas a otros; `/etc/ssh/ssh_config`) · **`sshd`** = el servidor (quién entra aquí; `/etc/ssh/sshd_config`).

Comando	Para qué
`ssh user@192.168.1.200`	Conectarse
`ssh -v user@IP`	Ver paso a paso la conexión (diagnóstico)
`ssh -o PubkeyAuthentication=no user@IP`	Probar que la contraseña está desactivada
`ssh-keygen -t ed25519`	Generar claves, **en el cliente**
`scp origen usuario@IP:/ruta/`	Copiar un fichero por SSH (`D:\tienda.sql user@IP:/home/user/`)
`sshd -t` · `sshd -T`	Validar la sintaxis · ver la configuración **final** que se aplicará

Clave pública en el servidor desde Windows (no tiene `ssh-copy-id`):

```
type %USERPROFILE%\.ssh\id_ed25519.pub | ssh user@IP "mkdir -p ~/.ssh && chmod 700 ~/.ssh && cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/authorized_keys"
```

Privada en el **cliente**, nunca sale de ahí; pública en `~/.ssh/authorized_keys` del servidor. Permisos: `~/.ssh` **700**, `authorized_keys` **600**.

Bastionado en `/etc/ssh/sshd_config.d/10-bastionado.conf` (debe acabar en **`.conf`**; en sshd **gana la primera** opción leída):

```
PermitRootLogin no
PasswordAuthentication no
AllowUsers user
```

Orden seguro: editar → `sshd -t` → `sshd -T | grep -Ei 'permitroot|password|allowusers'` → `systemctl restart ssh` → probar desde **otra** sesión **sin cerrar** la que tienes. `PermitRootLogin` vale por defecto `prohibit-password` (root sí podría entrar con clave).

Quién puede entrar: `AllowUsers` y `AllowGroups`

Son **filtros en serie (Y), no una suma (O)**. sshd los aplica en este orden y el usuario tiene que pasar **todos** los que existan, estén escritos en el orden y el fichero que sea:

1. `DenyUsers` : si aparece, fuera.
2. `AllowUsers` : si la línea existe y no aparece, fuera.
3. `DenyGroups` : si está en uno de esos grupos, fuera.
4. `AllowGroups` : si la línea existe y no está en ninguno, fuera.

Con `AllowUsers user` **y** `AllowGroups almacen` a la vez, nadie entra: `user` no está en `almacen` y `mrojas` no está en `AllowUsers`.

"Estos usuarios o los de este grupo" → grupo puente y solo `AllowGroups` :

```
groupadd accesossh
usermod -aG accesossh user
```

```
# /etc/ssh/sshd_config.d/...conf  (sin ninguna línea AllowUsers)
AllowGroups almacen accesossh
```

Opción	Cuándo
Solo `AllowGroups` + grupo puente	Lo normal: dar o quitar acceso = meter o sacar del grupo, sin tocar sshd.
Solo `AllowUsers` con todos listados	Cuando hay que limitar **por IP** (`usuario@192.168.1.177`): `AllowGroups` no puede. No entiende de grupos.
Las dos juntas	Solo si de verdad se quiere que se cumplan **ambas**.

- Nombres de grupo en minúsculas y sin espacios; `AllowGroups` distingue mayúsculas.
- Estas directivas **se acumulan entre ficheros**. Antes de recargar: `grep -ri allow /etc/ssh/sshd_config /etc/ssh/sshd_config.d/`.
- Recargar solo si la sintaxis es buena: `sshd -t && systemctl reload ssh`, y probar en **otra ventana** sin cerrar la actual.

`usermod -aG grupo usuario`: `-G` fija los grupos secundarios; `-a` los **añade** a los que ya tiene. Sin `-a` los **sustituye** (y el usuario puede perder `sudo`). Comprobar con `id usuario` antes y después; el grupo nuevo vale desde el siguiente inicio de sesión.

Instalar la clave de otro usuario con `PasswordAuthentication no`

Ese usuario no puede entrar con contraseña para copiar su clave (`Permission denied (publickey)`), así que se instala **desde la VM como root**.

1. En el PC, enviar la clave pública con un usuario que ya entra:

```
type C:\Users\si\.ssh\id_rsa.pub | ssh -p 2222 user@192.168.1.200 "cat > /tmp/mrojas.pub"
```

2. En la VM, como root:

```
install -d -m 700 -o mrojas -g mrojas /home/mrojas/.ssh
cat /tmp/mrojas.pub >> /home/mrojas/.ssh/authorized_keys
chown mrojas:mrojas /home/mrojas/.ssh/authorized_keys
chmod 600 /home/mrojas/.ssh/authorized_keys
rm /tmp/mrojas.pub
ls -la /home/mrojas/.ssh/
```

Línea	Qué hace
`install -d -m 700 -o mrojas -g mrojas .../.ssh`	Crea la carpeta (`-d`) con permisos `700` y dueño `mrojas` de una vez (= `mkdir` + `chmod` + `chown`).
`cat ... >> authorized_keys`	Añade la clave al fichero donde sshd mira qué claves pueden entrar. `>>` añade al final sin borrar otras.
`chown mrojas:mrojas ...`	Creado por root → hay que dárselo a `mrojas`.

Línea	Qué hace
`chmod 600 ...`	Solo `mrojas` lo lee y escribe.
`rm /tmp/mrojas.pub`	`/tmp` lo ve todo el mundo: no dejar copias sueltas.
`ls -la .../.ssh/`	Verificación: `drwx------ mrojas` y `-rw------- mrojas`.

3. Probar desde el PC, en otra ventana: `ssh -p 2222 mrojas@192.168.1.200` entra sin contraseña; `ssh -p 2222 root@192.168.1.200` es rechazado.

Si `.ssh` o `authorized_keys` tienen permisos más abiertos o son de otro usuario, sshd **ignora la clave sin decir por qué** (para que nadie cuele su clave en otra cuenta). Si una clave "no funciona", es lo primero que hay que mirar, junto con `journalctl -u ssh`.

En el cliente, el puerto va con **`-p 2222`**, nunca `host:2222` (lo tomaría como nombre de host).

Caso completo: dar acceso SSH con clave a un usuario nuevo

Ejemplo real con `avidal`. El usuario no puede entrar con contraseña (`PasswordAuthentication no`), así que **la clave la instala root dentro de la VM**. Hay dos máquinas: mira el prompt en cada paso (`C:\Users\si>` = PC · `root@srv-sistemas:...#` = VM como root).

Paso 0. En el PC: tener una clave. Mirar si ya existe; si no, crearla (Enter en todo):

```
dir %USERPROFILE%\.ssh
ssh-keygen -t ed25519
```

Salen dos ficheros: `id_ed25519` (privada, **nunca sale del PC**) e `id_ed25519.pub` (pública, la que se copia).

Paso 1. En la VM, como root: que el usuario pueda pasar el filtro de sshd.

```
grep -ri allow /etc/ssh/sshd_config /etc/ssh/sshd_config.d/
usermod -aG accesossh avidal
id avidal
```

Si el grupo ya está en `AllowGroups`, basta con meter al usuario: **no se toca sshd ni se recarga**. Solo si cambias la configuración: `sshd -t && systemctl reload ssh` (nunca `restart` sin `sshd -t`, y `daemon-reload` no sirve para esto).

Paso 2. En el PC: enviar la clave pública a `/tmp` con un usuario que YA entra (`user`).

```
type %USERPROFILE%\.ssh\id_ed25519.pub | ssh -p 2222 user@192.168.1.200 "cat
> /tmp/avidal.pub"
```

Entre comillas, **solo** guardar en `/tmp`. Si dejas `~/.ssh/authorized_keys`, la clave va a parar a `user`, no a `avidal`.

Paso 3. En la VM, como root: comprobar que ha llegado.

```
ls -l /tmp/avidal.pub
```

Si no existe, el paso 2 falló: no sigas.

Paso 4. En la VM, como root: instalarla con el dueño y los permisos del usuario.

```
install -d -m 700 -o avidal -g avidal /home/avidal/.ssh
cat /tmp/avidal.pub >> /home/avidal/.ssh/authorized_keys
chown avidal:avidal /home/avidal/.ssh/authorized_keys
chmod 600 /home/avidal/.ssh/authorized_keys
rm /tmp/avidal.pub
ls -la /home/avidal/.ssh/
```

Resultado bueno: `drwx------ avidal avidal .` y `-rw------- avidal avidal authorized_keys`.

Paso 5. Quitar el fin de línea de Windows (`\r`). Lo que pasa `type` llega con `\r\n`. Se ve con `cat -A` (la línea acaba en `^M$`):

```
cat -A /home/avidal/.ssh/authorized_keys
sed -i 's/\r$//' /home/avidal/.ssh/authorized_keys
ls -l /home/avidal/.ssh/authorized_keys
```

Paso 6. Probar desde una ventana NUEVA del PC, sin cerrar la que tienes.

```
ssh -p 2222 user@192.168.1.200
ssh -p 2222 avidal@192.168.1.200
ssh -p 2222 cnavarro@192.168.1.200
```

`user` entra · `avidal` entra **sin pedir contraseña** · `cnavarro` (fuera de los grupos) da `Permission denied (publickey)`. Si `avidal` no entra: `journalctl -u ssh --no-pager -n 20` y revisar permisos y dueño del paso 4.

Lo que me falló haciendo esto:

Lo que hice	Qué pasó	Lo correcto
`ssh avidal@...` sin `-p 2222`	`port 22: Connection refused`	`ssh -p 2222 ...`
Enviar la clave entrando como `avidal`	`Permission denied (publickey)`: aún no tiene clave	Enviarla con `user` a `/tmp`
Cambiar a `user` y dejar `~/.ssh/authorized_keys` en el comando	La clave se añadió (duplicada) a `user`	Entre comillas solo `cat > /tmp/avidal.pub`
`chmod 700 /home/avidal/.shh`	`No such file or directory`	Tabulador: si no completa, el nombre está mal
`mv /tmp/avidal.pub ... authorized_keys`	Sustituye en vez de añadir y deja el dueño de `/tmp`	`cat ... >> authorized_keys` + `chown` + `chmod`
`mv fichero >> destino`	`missing destination file operand`	`>>` es para la salida de un comando como `cat`, no para `mv`
`awk` para quitar la línea repetida	No la quitó: una tenía `\r`	`cat -A` para ver la diferencia, `sed -i 's/\r$//'`

Cómo comprobarlo: SSH

Qué compruebas	Comando	Qué debe salir
Sintaxis	`sudo sshd -t`	Nada

Qué compruebas	Comando	Qué debe salir
Lo que se va a aplicar	`sudo sshd -T | grep -Ei 'port|permitroot|password|allowusers|maxauth'`	Los valores esperados
Permisos de las claves	`ls -ld ~/.ssh` · `ls -l ~/.ssh/authorized_keys`	`drwx------` · `-rw-------`, del usuario
Entra con clave	Desde el PC: `ssh -v user@IP`	*Authenticated … using "publickey"*
Contraseña rechazada	`ssh -o PubkeyAuthentication=no user@IP`	*Permission denied (publickey)* sin pedir contraseña
Root y otros, rechazados	`ssh root@IP` · `ssh soporte@IP`	*Permission denied*
Por qué se rechazó	`sudo journalctl -u ssh -n 30`	*not listed in AllowUsers*
Escucha donde debe	`sudo ss -tlpn | grep sshd`	El puerto configurado

Firewall con nftables

`nftables` es el sistema; `nft`, el comando. Todo en **`/etc/nftables.conf`**.

Comando	Para qué
`sudo nft list ruleset`	Ver las reglas **activas** (en memoria)
`sudo nft -c -f /etc/nftables.conf`	**Validar** el fichero sin aplicarlo
`sudo nft -f /etc/nftables.conf`	Aplicar el fichero (atómico: si hay error, no cambia nada)
`sudo systemctl enable --now nftables`	Cargar el fichero en cada arranque

```
#!/usr/sbin/nft -f
flush ruleset

table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;
        ct state { established, related } accept
        iif lo accept
        meta l4proto { icmp, ipv6-icmp } accept
        tcp dport { 22, 80 } accept
    }
    chain forward {
        type filter hook forward priority filter;
    }
    chain output {
        type filter hook output priority filter;
    }
}
```

Anatomía de una regla: condiciones (dónde mirar + valor) y una acción. Se leen de **arriba abajo**; la primera que coincide decide; si ninguna, la **política**.

Pieza	Mira
`inet`	IPv4 **e** IPv6 a la vez
`iif lo`	La interfaz de entrada (`lo` = la máquina consigo misma)
`ip saddr 192.168.1.177`	IP de origen (IPv4)
`ip protocol` · `ip6 nexthdr` · `meta l4proto`	Protocolo de capa 4: en IPv4 · en IPv6 · en los dos
`tcp dport 22`	Puerto de destino TCP
`ct state established,related`	Respuestas a conexiones ya abiertas
`{ a, b }`	Cualquiera de esos valores

Por qué cada regla: sin `lo` , Nginx no llega a la aplicación en `127.0.0.1` · sin todo el ICMPv6, IPv6 deja de funcionar (NDP sustituye a ARP) · sin `established` , no llegan las respuestas.

Cuidado: editar el fichero **no aplica nada** hasta `nft -f`; adapta los ejemplos de internet a tu interfaz; nunca pongas la política `drop` antes de permitir SSH con comandos sueltos; abre solo lo que se usa (sin HTTPS configurado, el 443 cerrado).

Cómo comprobarlo: firewall

Qué compruebas	Comando	Qué debe salir
Sintaxis del fichero	`sudo nft -c -f /etc/nftables.conf`	Nada
Reglas activas	`sudo nft list ruleset`	Las de tu fichero (si no, falta `nft -f`)
Se carga al arrancar	`systemctl is-enabled nftables` · `status nftables`	`enabled` · *active (exited)*
Lo local funciona	`curl -m 5 http://localhost/app/`	Responde (sin `iif lo`, se cuelga)
Lo permitido entra	Desde el PC: `ssh` y `curl http://IP/`	Funcionan
Lo demás se bloquea	En la VM `python3 -m http.server 9000`; desde el PC `curl -m 5 http://IP:9000`	*timed out*
Sobrevive al reinicio	`reboot` → `nft list ruleset`	Las mismas reglas

Nginx y proxy inverso

En resumen, para publicar una web con Nginx:

1. **El contenido:** la carpeta `/var/www/nombre` con su `index.html` y el resto de ficheros. La web es **la carpeta entera**: cada ruta es un fichero (`root` + ruta: `/contacto.html` → `/var/www/nombre/contacto.html`); el `index.html` es el que se entrega al pedir la carpeta (`/`).
2. **La configuración:** un fichero en `/etc/nginx/sites-available/` que dice **dónde escucha** (`listen`), **de qué carpeta sirve** (`root`, `index`), **dónde guarda los logs** y **qué hace con cada ruta** (`location`: servir ficheros o pasarla a otra aplicación). La

línea `root` es la que "conecta" la configuración con el HTML.

3. **Activarla:** enlace en `sites-enabled` → `nginx -t` → `systemctl reload nginx`.
4. **Abrir el puerto** en el firewall si es nuevo y **comprobar** con `curl`.

Nginx no "muestra" la web: **entrega** el contenido de los ficheros y el navegador lo muestra.

Qué es: un **servidor web**, el programa que recibe las peticiones HTTP y decide qué responder. Hace dos cosas: **servir ficheros** (una carpeta con `index.html`) y hacer de **proxy inverso** (pasar la petición a otra aplicación, como la API en el 8000, y devolver su respuesta). Va delante de las aplicaciones como **única puerta de entrada**: ellas quedan en `127.0.0.1`, y Nginx añade logs, contraseñas, HTTPS…

Un servicio, muchos sitios. Hay **un solo servicio** (`nginx.service`, con `systemctl`). Cada **sitio** (`web`, `intranet`) **no es un servicio**: es un fichero de **configuración** que Nginx lee. Para un sitio nuevo no se toca systemd: se añade configuración y se recarga.

Pieza	Qué es	Dónde
El servicio	El programa Nginx funcionando	`systemctl status nginx`
Configuración general	Ajustes comunes; incluye los sitios activos	`/etc/nginx/nginx.conf`
Sitios disponibles	La configuración de cada sitio (el "almacén")	`/etc/nginx/sites-available/`
Sitios activos	**Enlaces** a los que se cargan	`/etc/nginx/sites-enabled/`
Contenido	Las páginas de cada sitio	`/var/www/nombre/`
Logs	Peticiones y errores	`/var/log/nginx/`

Comando / ruta	Para qué
`/etc/nginx/sites-available/web`	Donde se **escribe** la configuración del sitio
`sudo ln -s /etc/nginx/sites-available/web /etc/nginx/sites-enabled/`	**Activar** el sitio (enlace)
`sudo rm /etc/nginx/sites-enabled/default`	**Desactivar** el sitio por defecto (solo el enlace)
`/var/www/web/index.html`	El contenido de la web

Comando / ruta	Para qué
`sudo nginx -t`	Validar la configuración
`sudo systemctl reload nginx`	Aplicar sin cortar (si falla, sigue con la configuración anterior)
`/var/log/nginx/access.log` · `error.log`	Peticiones recibidas · errores

```
server {
    listen 80;
    listen [::]:80;
    server_name _;
    root /var/www/web;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /app/ {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

- `root` apunta a la **carpeta**; `index`, al fichero.
- `server_name` elige el sitio según el **nombre** que pide el navegador (cabecera `Host`); `_` = cualquiera.
- `proxy_pass` **con** `/` final quita `/app/` de la ruta; **sin** `/`, la pasa entera.
- Las cabeceras `X-Real-IP` y `X-Forwarded-For` pasan la IP real del cliente a la aplicación.

Código	Significado
200	Correcto
403	Nginx no puede leer los ficheros
404	No existe esa ruta

Código	Significado
502	La aplicación de detrás **no responde** (parada o en otro puerto)
504	La aplicación tarda demasiado (o la bloquea un `drop`)

Aplicación de prueba: `python3 -m http.server 8000 --bind 127.0.0.1`.

Crear un sitio paso a paso: la intranet

1. **Contenido:** `mkdir /var/www/intranet` y la página dentro (`index.html`, o el nombre que pongas en `index`).
2. **Configuración:** `nano /etc/nginx/sites-available/intranet`.
3. **Activar** (Nginx solo carga lo que hay en `sites-enabled`): `ln -s /etc/nginx/sites-available/intranet /etc/nginx/sites-enabled/`. Para desactivarlo, se borra el enlace (la configuración se conserva).
4. **Validar:** `nginx -t`.
5. **Aplicar:** `systemctl reload nginx` (sin cortar conexiones; si hay error, sigue con la anterior).
6. **Firewall:** abrir el puerto si es nuevo (`nft -c -f` → `nft -f`, y comprobar Docker).
7. **Comprobar:** `curl http://localhost:8080/`, desde el PC, y sus logs.

Ejemplo de página, `/var/www/intranet/index.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Intranet</title>
</head>
<body>
    <h1>Intranet de la biblioteca</h1>
    <p>Bienvenido.</p>
</body>
</html>
```

Línea	Qué hace
`<!DOCTYPE html>`	Indica que es una página HTML moderna
`<html lang="es">`	Empieza la página; el idioma es español
`<head>` … `</head>`	Información **sobre** la página, que no se ve en el cuerpo
`<meta charset="utf-8">`	Codificación de caracteres: sin ella, las tildes y la ñ salen mal
`<title>Intranet</title>`	El texto de la **pestaña** del navegador
`<body>` … `</body>`	Lo que **se ve** en la página
`<h1>…</h1>`	Un **título** grande
`<p>…</p>`	Un **párrafo**

Para una prueba rápida basta una línea: `echo "<h1>Intranet de la biblioteca</h1>" > /var/www/intranet/index.html`.

Los logs **no se crean a mano**: Nginx los crea al recargar. Creados por root con `nano` quedarían `root:root` y el grupo `adm` no podría leerlos.

Contenido de `/etc/nginx/sites-available/intranet`:

```
server {
    listen 8080;
    listen [::]:8080;
    server_name _;

    root /var/www/intranet;
    index intranet.html;

    access_log /var/log/nginx/intranet.access.log;
    error_log  /var/log/nginx/intranet.error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Línea	Qué significa
`server { … }`	**Un sitio**: todo lo que va entre llaves es su configuración
`listen 8080;` · `listen [::]:8080;`	En qué **puerto** atiende, por IPv4 · por IPv6
`server_name _;`	Para qué **nombre** responde (decide entre varios sitios en el mismo puerto). `_` = cualquiera
`root /var/www/intranet;`	La **carpeta** de los ficheros: `/foto.png` → `/var/www/intranet/foto.png`
`index intranet.html;`	Qué fichero se sirve al pedir una carpeta (`/`). Por defecto, `index.html`
`access_log …;`	Dónde se apunta **cada petición** (quién, cuándo, qué, código)
`error_log …;`	Dónde se apuntan los **errores**
`location / { … }`	Qué hacer con las rutas que empiezan por `/` (las que no tengan un `location` más concreto)
`try_files $uri $uri/ =404;`	Busca un **fichero** con ese nombre; si no, una **carpeta**; si tampoco, **404**
`location /api/ { proxy_pass http://127.0.0.1:8001/; }`	Proxy: las rutas `/api/…` van a la app del 8001. La **barra final** quita el `/api/`
`proxy_set_header X-Real-IP $remote_addr;`	Pasa a la app la **IP real** del cliente (si no, solo vería `127.0.0.1`)

Cuidado: un `proxy_pass` al **mismo puerto** del sitio (`8080` dentro del sitio del `8080`) se llama a sí mismo en bucle.

Cómo comprobarlo: Nginx

Qué compruebas	Comando	Qué debe salir
Sintaxis	`sudo nginx -t`	*syntax is ok* y *test is successful*

Qué compruebas	Comando	Qué debe salir
Sitios activos	`ls -l /etc/nginx/sites-enabled/`	Solo tu enlace (`web -> ...`)
La web responde	`curl -I http://localhost/`	`200 OK`
El proxy funciona	`curl http://localhost/app/`	La respuesta de la aplicación
La app solo en localhost	`sudo ss -tlpn | grep 8000` · desde el PC `curl -m 5 http://IP:8000`	`127.0.0.1:8000` · falla
Si la app cae	Parar la app → `curl -I http://localhost/app/`	`502 Bad Gateway`
Desde fuera	Desde el PC: `curl http://IP/`	La página
Peticiones recibidas	`sudo tail /var/log/nginx/access.log`	La IP del PC y el código
Errores	`sudo tail /var/log/nginx/error.log`	*connect() failed … upstream* si falla la app

MariaDB

Dentro de MariaDB todo es **SQL** y cada orden acaba en `;` (si ves `->`, falta el `;`; Ctrl+C cancela). Nombres de tablas y columnas **sin comillas**; textos y fechas **con comillas simples**.

El servidor (desde Linux)

Comando	Para qué
`sudo apt install mariadb-server`	Instalar (servicio `mariadb`, puerto 3306)
`sudo mariadb-secure-installation`	Asegurar: quitar anónimos, `test` y root remoto
`sudo mariadb`	Entrar como root (sin contraseña: `unix_socket`)
`mariadb -u lector -p tienda`	Entrar como otro usuario, en una base de datos
`sudo mariadb -e "SHOW DATABASES;"`	Ejecutar una orden sin entrar

Comando	Para qué
`sudo mariadb tienda < fichero.sql`	**Importar** un fichero SQL
`grep -r bind-address /etc/mysql/`	En qué IP escucha (`50-server.cnf` ; aquí **gana la última** opción leída)

Moverse por el servidor

Orden	Para qué
`SHOW DATABASES;`	Ver las bases de datos
`CREATE DATABASE tienda;` · `DROP DATABASE tienda;`	Crear · borrar una base de datos entera
`USE tienda;`	Entrar en ella (como un `cd`)
`SHOW TABLES;`	Ver sus tablas
`DESCRIBE clientes;`	Ver las columnas de una tabla
`SHOW CREATE TABLE pedidos\G`	Cómo se creó, con sus claves foráneas (`\G` = en vertical)

Sobre las filas (los datos)

Orden	Para qué
`SELECT nombre FROM clientes WHERE ciudad = 'Madrid';`	**Leer**
`INSERT INTO clientes VALUES (9, 'Nombre', 'Valencia', '2026-11-03');`	**Añadir** una fila
`UPDATE clientes SET ciudad = 'Madrid' WHERE id = 9;`	**Modificar** (sin `WHERE` , cambia **todas**)
`DELETE FROM clientes WHERE id = 9;`	**Borrar filas** (sin `WHERE` , borra **todas**)

Plantilla de consulta: `SELECT col, SUM(x) AS total FROM tabla WHERE filtro_filas GROUP BY col HAVING filtro_grupos ORDER BY total DESC LIMIT 5;`. Año de una fecha: `YEAR(fecha)` (en cualquier motor, `SUBSTRING(fecha, 1, 4)`). Contar filas de varias tablas: `SELECT 'clientes', COUNT(*) FROM clientes UNION ALL SELECT ...;`. Tabla de otra base de datos: `tienda_restore.clientes`.

Sobre las tablas (la estructura)

Orden	Para qué
`CREATE TABLE nombre (id INT PRIMARY KEY, ...);`	Crear una tabla
`ALTER TABLE nombre ADD columna TEXT;`	Cambiar sus columnas
`TRUNCATE TABLE nombre;`	Vaciarla entera (la tabla sigue)
`DROP TABLE nombre;`	**Borrar la tabla** entera, estructura incluida

`DELETE` borra filas · **`TRUNCATE`** vacía la tabla · **`DROP`** elimina la tabla.

Plantilla: dos tablas con clave foránea y CHECK

Ejemplo del observatorio. Se crea primero la tabla **a la que se apunta** (`estaciones`) y después la que apunta (`lecturas`).

```
CREATE DATABASE observatorio;
USE observatorio;

CREATE TABLE estaciones (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL UNIQUE,
  ubicacion VARCHAR(100)
);

CREATE TABLE lecturas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  estacion INT NOT NULL,
  fecha DATETIME NOT NULL,
  temperatura DECIMAL(4,1),
  humedad TINYINT UNSIGNED,
```

```
  CONSTRAINT chk_humedad CHECK (humedad BETWEEN 0 AND 100),
  CONSTRAINT fk_lecturas_estacion FOREIGN KEY (estacion) REFERENCES
estaciones(id)
);

DESCRIBE lecturas;
SHOW CREATE TABLE lecturas;   -- deben salir el CHECK y la FOREIGN KEY
```

Pide el enunciado	Tipo	Ojo
Fecha	`DATE`	Solo día: `'2026-11-10'`
Fecha **y hora**	`DATETIME`	`'2026-11-10 09:30:00'`; con `DATE` la hora se pierde
Un decimal (21.5)	`DECIMAL(4,1)`	4 cifras en total, 1 decimal: de -999.9 a 999.9
Entero de 0 a 100	`TINYINT UNSIGNED` + `CHECK`	El `CHECK` es lo que impide el 101
Único y obligatorio	`NOT NULL UNIQUE`	

Cada columna = nombre + tipo + opciones. Sin tipo (`id AUTO_INCREMENT`), sale `Unknown data type: 'AUTO_INCREMENT'`.

En `FOREIGN KEY (columna)`, la columna es de ESTA tabla, con el nombre exacto con que la has creado. Si no coincide (`estacion_id` y la columna se llama `estacion`), sale `ERROR 1072: Key column ... doesn't exist`.

Comprobar las restricciones:

```
INSERT INTO estaciones (nombre) VALUES ('Puerto');
-- Duplicate entry si ya existe
INSERT INTO lecturas (estacion, fecha, humedad) VALUES (999, NOW(), 50);    -
- error de clave foránea
INSERT INTO lecturas (estacion, fecha, humedad) VALUES (1, NOW(), 150);     -
- CONSTRAINT chk_humedad failed
```

Media por estación con el nombre (JOIN + GROUP BY):

```
SELECT e.nombre, ROUND(AVG(l.temperatura),1) AS media, COUNT(*) AS lecturas
FROM lecturas l JOIN estaciones e ON l.estacion = e.id
GROUP BY e.id, e.nombre;
```

Usuarios y permisos

Un usuario es `'nombre'@'host'`: `'lector'@'localhost'` (solo desde la máquina) y `'lector'@'%'` (desde cualquier sitio) son **usuarios distintos**. Son independientes de los usuarios de Linux.

Orden	Para qué
`CREATE USER 'lector'@'localhost' IDENTIFIED BY 'clave';`	Crear (**siempre con contraseña**)
`ALTER USER 'lector'@'localhost' IDENTIFIED BY 'nueva';`	Cambiar la contraseña
`DROP USER 'lector'@'localhost';`	Borrar el usuario
`GRANT SELECT ON tienda.* TO 'lector'@'localhost';`	Dar permisos
`REVOKE INSERT ON tienda.* FROM 'gestor'@'localhost';`	Quitar permisos
`SHOW GRANTS FOR 'lector'@'localhost';` · `SHOW GRANTS;`	Ver sus permisos (como `sudo -l`)
`SELECT User, Host FROM mysql.user;`	Ver todos los usuarios

Familia	Permisos
Datos (filas)	`SELECT` leer · `INSERT` añadir · `UPDATE` modificar · `DELETE` borrar filas
Estructura (tablas)	`CREATE` · `DROP` · `ALTER` · `INDEX`
Todo	`ALL PRIVILEGES`

Alcance: `*.*` todo el servidor · `tienda.*` una base de datos · `tienda.pedidos` una tabla. `USAGE` = sin permisos (solo conectarse).

Todos los permisos que se pueden dar

Se dan con `GRANT permiso ON alcance TO 'usuario'@'host';` y se quitan con `REVOKE permiso ON alcance FROM 'usuario'@'host';`. Pueden ir varios separados por comas.

Sobre los datos (los más usados):

Permiso	Qué permite
`SELECT`	**Leer** filas
`INSERT`	**Añadir** filas
`UPDATE`	**Modificar** filas
`DELETE`	**Borrar** filas
`ALL PRIVILEGES` (o `ALL`)	**Todos** los permisos del alcance indicado

Sobre la estructura:

Permiso	Qué permite
`CREATE`	Crear bases de datos y tablas
`DROP`	**Eliminar** bases de datos y tablas
`ALTER`	Modificar la estructura de una tabla (columnas, tipos…)
`INDEX`	Crear y borrar índices
`REFERENCES`	Crear claves foráneas
`CREATE TEMPORARY TABLES`	Crear tablas temporales
`CREATE VIEW` · `SHOW VIEW`	Crear y ver vistas
`TRIGGER`	Crear disparadores
`EVENT`	Crear tareas programadas dentro de la base de datos

Programación dentro de la base de datos:

Permiso	Qué permite
`CREATE ROUTINE`	Crear procedimientos y funciones
`ALTER ROUTINE`	Modificarlos o borrarlos
`EXECUTE`	**Ejecutarlos**

De administración (casi siempre solo en `*.*` y para administradores):

Permiso	Qué permite
`LOCK TABLES`	Bloquear tablas (lo necesita `mariadb-dump`, salvo con `--single-transaction`)
`PROCESS`	Ver las consultas que se están ejecutando
`RELOAD`	Recargar permisos y vaciar cachés
`SHOW DATABASES`	Ver la lista de todas las bases de datos
`FILE`	Leer y escribir ficheros del servidor desde SQL
`SHUTDOWN`	Apagar el servidor
`SUPER`	Muchas operaciones administrativas (en versiones nuevas se divide en permisos más concretos)
`CREATE USER`	Crear, modificar y borrar usuarios
`REPLICATION SLAVE` · `REPLICATION CLIENT`	Para la replicación entre servidores

Especiales:

Permiso	Qué significa
`USAGE`	**Ningún** permiso: el usuario existe y puede conectarse, nada más (es lo que sale en `SHOW GRANTS` aunque no tenga nada)

Permiso	Qué significa
`GRANT OPTION`	Puede **dar a otros** los permisos que él tiene. No se incluye en `ALL PRIVILEGES`: se añade con `WITH GRANT OPTION` al final del `GRANT`

Combinaciones habituales:

Perfil	Permisos
Solo lectura	`GRANT SELECT ON biblioteca.* TO ...;`
Aplicación que lee y escribe	`GRANT SELECT, INSERT, UPDATE, DELETE ON biblioteca.* TO ...;`
Solo añadir y modificar en una tabla	`GRANT SELECT ON biblioteca.* TO ...;` + `GRANT INSERT, UPDATE ON biblioteca.prestamos TO ...;`
Copias de seguridad	`GRANT SELECT, LOCK TABLES, SHOW VIEW, EVENT, TRIGGER ON biblioteca.* TO ...;`
Administrador de una base	`GRANT ALL PRIVILEGES ON biblioteca.* TO ...;`

Los permisos **se suman**: el usuario tiene la unión de todos sus `GRANT`. Para quitar solo uno, `REVOKE` de ese permiso en ese alcance. Si el alcance del `REVOKE` no coincide con el del `GRANT` (por ejemplo, un `GRANT` en `biblioteca.*` y un `REVOKE` en `biblioteca.libros`), **no se resta**.

Ver y comprobar: `SHOW GRANTS FOR 'usuario'@'host';` muestra lo que tiene. Después de cambiar permisos con `GRANT` o `REVOKE` **no hace falta** `FLUSH PRIVILEGES` (solo si se modifican a mano las tablas `mysql.*`). El usuario conectado **recibe los cambios** de base de datos y tabla en su siguiente consulta, pero los de `*.*` solo al **reconectar**.

Copias de seguridad

Comando	Para qué
`sudo mariadb-dump --single-transaction tienda > copia.sql`	Copia coherente sin bloquear

Comando	Para qué
`sudo mariadb-dump --single-transaction tienda | gzip > copia.sql.gz`	Copia comprimida
`sudo mariadb tienda_restore < copia.sql`	Restaurar
`sudo zcat copia.sql.gz | sudo mariadb tienda_restore`	Restaurar una copia comprimida

Una copia que no has **restaurado y comprobado** (contando filas) no es una copia.

Errores frecuentes

Error	Significa
1064	Sintaxis mal escrita (a menudo, falta un `;`)
1054 / 1146	No existe esa columna / esa tabla (nombre mal escrito; texto sin comillas)
1045	No te deja entrar: usuario, contraseña o host
1142	Has entrado, pero **no tienes permiso**
1452 / 1451	Clave foránea: fila hija sin madre / no se puede borrar la madre
2002	No hay servidor escuchando en esa dirección

Acceso remoto: tres capas lo impiden: firewall (3306 cerrado) · `bind-address = 127.0.0.1` · usuarios `@'localhost'`. Para administrar en remoto, entrar por SSH.

Cómo comprobarlo: MariaDB

Qué compruebas	Comando	Qué debe salir
El servicio	`systemctl status mariadb --no-pager`	*active (running)*, *enabled*
Dónde escucha	`sudo ss -tlpn | grep 3306`	`127.0.0.1:3306`

Qué compruebas	Comando	Qué debe salir
Sin anónimos ni root remoto	`SELECT User, Host FROM mysql.user;`	Ningún `User` vacío; `root` solo `localhost`
Sin base de pruebas	`SHOW DATABASES;`	No aparece `test`
Datos importados	`SELECT COUNT(*) FROM clientes;` (cada tabla)	Las filas esperadas
Claves foráneas	`SHOW CREATE TABLE pedidos\G`	`CONSTRAINT ... FOREIGN KEY`
Permisos de un usuario	`SHOW GRANTS FOR 'lector'@'localhost';`	Solo lo que debe, y `IDENTIFIED BY PASSWORD`
Lo que no puede hacer	`mariadb -u lector -p tienda` → `UPDATE ...`	`ERROR 1142`
No accesible desde fuera	`mariadb -h 192.168.1.200 -u lector -p` · desde el PC `curl -m 5 telnet://IP:3306`	`ERROR 2002` · *timed out*
La copia sirve	Restaurar en `tienda_restore` y comparar `COUNT(*)`	Mismas cifras

Docker: el Dockerfile

El `Dockerfile` es la **receta** de una imagen. Va en la **carpeta del proyecto**, junto al código, con ese nombre exacto (D mayúscula, sin extensión):

```
/srv/catalogo/
├── app.py              ← el código de la API
├── requirements.txt    ← las librerías que necesita
├── Dockerfile          ← la receta de la imagen
├── compose.yaml        ← la aplicación completa (api + db)
├── .env                ← contraseñas (chmod 600)
└── init/01-tienda.sql  ← datos iniciales de la base de datos
```

Contenido de `/srv/catalogo/Dockerfile`:

```
FROM python:3.12.7-slim

WORKDIR /app

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

RUN useradd --no-create-home --shell /usr/sbin/nologin appuser

COPY app.py .

USER appuser

EXPOSE 8000

CMD ["python", "app.py"]
```

Línea	Qué hace
`FROM python:3.12.7-slim`	Imagen **base**: Python ya instalado. `slim` = reducida; versión **fija**, nunca `latest`
`WORKDIR /app`	Carpeta de trabajo **dentro** del contenedor (la crea y hace `cd`)
`ENV PYTHONDONTWRITEBYTECODE=1`	Python no crea ficheros `.pyc`, inútiles en un contenedor
`ENV PYTHONUNBUFFERED=1`	Los mensajes salen al momento en `docker logs`
`COPY requirements.txt .`	Copia la lista de librerías del proyecto a `/app`
`RUN pip install --no-cache-dir -r requirements.txt`	Instala las librerías **al construir**; sin caché, la imagen ocupa menos
`RUN useradd --no-create-home --shell /usr/sbin/nologin appuser`	Crea un usuario sin carpeta ni sesión para ejecutar la app

Línea	Qué hace
`COPY app.py .`	Copia el código **después** de las librerías. Queda de **root**: la app no puede modificarlo
`USER appuser`	A partir de aquí, todo se ejecuta como `appuser`, **no root**
`EXPOSE 8000`	**Documenta** el puerto. No abre nada: se publica con `ports` o `-p`
`CMD ["python", "app.py"]`	Lo que se ejecuta **al arrancar** el contenedor

`RUN` frente a `CMD`: `RUN` se ejecuta **una vez, al construir**; `CMD`, **cada vez que arranca** un contenedor.

El orden importa (caché de capas): cada instrucción es una capa. Si solo cambias `app.py`, Docker reutiliza (`CACHED`) la capa de `pip install` y reconstruye en segundos. Por eso las dependencias van **antes** que el código.

Comando	Para qué
`sudo docker build -t catalogo:1.0 .`	Construir (el `.` es el **contexto**: solo puede copiar ficheros de esta carpeta)
`sudo docker images`	Ver las imágenes
`sudo docker run --rm catalogo:1.0 id`	Comprobar con qué usuario se ejecuta (sustituye el `CMD`)
`sudo docker run --rm catalogo:1.0 ls -l /app`	Comprobar de quién es el código
`sudo docker compose up -d --build`	Construir y arrancar la aplicación completa
`sudo docker compose ps` · `logs -f api` · `down`	Estado · logs · parar (`down -v` **borra los datos**)

Dentro del contenedor la app escucha en `0.0.0.0` (con `127.0.0.1` ni Docker llegaría a ella); la restricción a localhost se hace al publicar: `127.0.0.1:8000:8000`. Docker se salta la cadena `input` del firewall, así que nunca publicar puertos de más.

Cómo comprobarlo: Docker

Qué compruebas	Comando	Qué debe salir
Docker funciona	`sudo docker run --rm hello-world`	*Hello from Docker!*
Configuración de Compose	`sudo docker compose config`	Sin errores y con las variables sustituidas
Estado	`sudo docker compose ps`	`api` *Up*; `db` *Up (healthy)*
Puertos	Columna PORTS de `ps`	`api`: `127.0.0.1:8000->8000`; `db`: `3306/tcp` sin flecha
Usuario de la imagen	`sudo docker run --rm catalogo:1.0 id`	`appuser`, no root
La API responde	`curl http://127.0.0.1:8000/salud`	`{"estado":"ok"}`
Permisos en la base de datos	`sudo docker compose exec db mariadb -u root -p -e "SHOW GRANTS FOR 'api_lectura'@'%';"`	Solo `SELECT`
No accesible desde fuera	Desde el PC: `curl -m 5 telnet://IP:8000` y `:3306`	Fallan
Logs	`sudo docker compose logs -f api`	Las peticiones
Los datos persisten	Cambiar un dato → `down` (sin `-v`) → `up -d` → consultar	El cambio sigue
Se levanta si falla	`sudo docker top catalogo-api` → `sudo kill -9 PID` → `ps`	Vuelve a *Up*

Proyecto API + Compose: orden de creación

Cada fichero depende de los anteriores, así que se crean en este orden y se **comprueba cada uno** antes de pasar al siguiente. Todo en la carpeta del proyecto (por ejemplo `/srv/biblioteca/`).

```
/srv/biblioteca/
├── app.py
├── requirements.txt
├── Dockerfile
├── .dockerignore
├── .env                        (600)
├── initdb/
│   ├── 01-biblioteca.sql
│   └── 02-lector.sh
└── compose.yaml
```

#	Fichero	Para qué sirve	Cómo se comprueba
1	`app.py`	**El programa**: qué rutas tiene la API, qué consulta lanza y qué JSON devuelve. Corre **dentro** del contenedor y no sabe nada de Docker.	Se prueba con la imagen (paso 3).
2	`requirements.txt`	Las librerías de Python que usa `app.py` (`Flask`, `mysql-connector-python`, `gunicorn`), una por línea.	`cat requirements.txt`
3	`Dockerfile`	**La receta de la imagen**: imagen base fija, `requirements.txt` + `pip install` **antes** de copiar el código (caché), `useradd` + `USER` al final, `CMD` con gunicorn.	`docker build -t nombre:1.0 . && docker run --rm nombre:1.0 id` → no root · `... ls -l /app` → ficheros de root
4	`initdb/01-...sql`	El volcado de la base de datos (`mariadb-dump biblioteca > initdb/01-biblioteca.sql`). MariaDB lo carga **solo la primera vez**, con el volumen vacío.	`grep -c INSERT initdb/01-*.sql`

#	Fichero	Para qué sirve	Cómo se comprueba
5	`initdb/02-lector.sh`	Crea el usuario de **solo lectura** (`'lector'@'%'`, `GRANT SELECT`). Es `.sh` para leer la contraseña de una variable, no escribirla. Se ejecuta **después** del `01` (orden alfabético).	`bash -n initdb/02-lector.sh`
6	`.env`	Las contraseñas (`MARIADB_ROOT_PASSWORD`, `LECTOR_PASSWORD`), sin espacios ni comillas. Así no van en `compose.yaml`.	`chmod 600 .env` → `ls -la` muestra `-rw-------`
7	`.dockerignore`	Excluye `.env` e `initdb/` del contexto del build: **seguridad**, para que no lleguen a Docker ni a la imagen.	`cat .dockerignore`
8	`compose.yaml`	**La organización**: qué servicios hay (`db`, `api`), imagen de cada uno, puertos, volúmenes, variables, `healthcheck`, `depends_on`, `restart`. Lo lee `docker compose` **en la VM**.	`docker compose config --quiet && echo OK`
9	Nginx (`sites-available/intranet`)	`location /api/` con `proxy_pass http://127.0.0.1:8001/;` para publicar hacia fuera una API que solo escucha en localhost.	`nginx -t && systemctl reload nginx`

`app.py` frente a `compose.yaml`: `app.py` dice **qué hace** la API; `compose.yaml`, **cómo se monta** el despliegue. Solo se tocan en las variables: Compose pasa `DB_HOST: db`, `DB_USER`, `DB_PASSWORD`… y `app.py` las lee con `os.getenv`.

Levantar y comprobar:

```
docker compose up -d --build
docker compose ps                 # db → healthy · api → Up (el tiempo sube,
no vuelve a 0)
docker compose logs api           # si algo falla, el motivo está aquí
curl -i http://127.0.0.1:8001/libros
curl -i http://localhost:8080/api/libros     # tras el paso 9: Server: nginx
```

Trampas vistas en la práctica:

- `docker run` de una imagen sin `docker build` antes → `pull access denied`: busca en Docker Hub una imagen que no existe.
- `USER` antes del `pip install` → `Permission denied`: el usuario no root no puede instalar paquetes.
- Si el `build` falla, `docker run` usa la **imagen anterior** con la misma etiqueta: encadena con `&&`.
- Un `curl` justo tras arrancar da `Connection reset by peer`: gunicorn aún no escucha; mira `docker logs` y espera a `Listening at`.
- Cambias `initdb/` y no se nota: solo se ejecuta con el volumen vacío → `docker compose down -v` (borra los datos) y `up` otra vez.
- `proxy_pass` **sin** barra final pasa `/api/libros` tal cual y la API da 404; **con** barra, llega como `/libros`.
- `"8001:8000"` publica en todas las interfaces y Docker **se salta tu firewall**; usa siempre `"127.0.0.1:8001:8000"`.

El código de `app.py` explicado línea a línea está justo debajo.

API de la biblioteca (`app.py`), línea a línea

Una API en Flask con tres rutas (`/salud`, `/libros`, `/libros/<id>`) que lee de MariaDB. En el contenedor la sirve **gunicorn** en el puerto 8000; la API **no sabe nada de Docker**: todo lo que cambia entre entornos (host, usuario, contraseña) le llega por **variables de entorno**.

1. Importaciones

```
import os
from decimal import Decimal
from datetime import date, datetime

import mysql.connector
from flask import Flask, jsonify

app = Flask(__name__)
```

Línea	Qué hace
`import os`	Para leer variables de entorno (`os.getenv` , `os.environ`).
`from decimal import Decimal`	MariaDB devuelve `DECIMAL` como `Decimal` , que JSON no sabe escribir; se importa para convertirlo.
`from datetime import date, datetime`	Igual con `DATE` / `DATETIME` (las fechas de `prestamos`).
`import mysql.connector`	El conector para hablar con MariaDB (paquete `mysql-connector-python` en `requirements.txt`).
`from flask import Flask, jsonify`	`Flask` crea la aplicación; `jsonify` convierte un dict o lista en respuesta JSON con su cabecera `Content-Type: application/json` .
`app = Flask(__name__)`	Crea la aplicación. Este `app` es el que arranca gunicorn con `app:app` (fichero `app.py` , variable `app`).

2. Configuración por variables de entorno

```
DB_CONFIG = {
    "host": os.getenv("DB_HOST", "db"),
    "port": int(os.getenv("DB_PORT", "3306")),
    "user": os.environ["DB_USER"],
    "password": os.environ["DB_PASSWORD"],
    "database": os.getenv("DB_NAME", "biblioteca")
}
```

Línea	Qué hace
`os.getenv("DB_HOST", "db")`	Lee la variable; si no existe usa `"db"`, **el nombre del servicio en Compose**. Dentro de un contenedor, `localhost` es el propio contenedor, nunca la base de datos.
`int(os.getenv("DB_PORT", "3306"))`	Las variables siempre llegan como texto: el puerto se pasa a número con `int()`.
`os.environ["DB_USER"]`	Sin valor por defecto a propósito: si falta, la API **falla al arrancar** (`KeyError`) en vez de conectar como root sin que nadie se entere.
`os.environ["DB_PASSWORD"]`	La contraseña nunca va en el código ni en `compose.yaml`: llega desde `.env` o un secreto.
`os.getenv("DB_NAME", "biblioteca")`	Base de datos a la que se conecta.

Tu primera versión tenía `"mysql"` y `"root"` como valores por defecto: con Compose el host es `db` y el usuario debe ser el de **solo lectura**.

3. Funciones auxiliares

```
def conectar_db():
    return mysql.connector.connect(**DB_CONFIG)


def convertir_json(valor):
    if isinstance(valor, Decimal):
        return float(valor)
    if isinstance(valor, (date, datetime)):
        return valor.isoformat()
    return valor


def normalizar_libro(libro):
    return {clave: convertir_json(valor) for clave, valor in libro.items()}
```

Línea	Qué hace
`mysql.connector.connect(**DB_CONFIG)`	Abre una conexión nueva. `**` "desempaqueta" el diccionario: equivale a `connect(host=..., port=..., user=...)`.
`isinstance(valor, Decimal)` → `float(valor)`	Convierte `Decimal` en número normal para JSON.
`isinstance(valor, (date, datetime))` → `isoformat()`	Convierte una fecha en texto `2026-11-09`.
`return valor`	Lo demás (int, str, None) ya es válido en JSON.
`{clave: convertir_json(valor) for ...}`	Recorre cada columna del libro y aplica la conversión. `anio` (tipo `YEAR`) llega como `int`.

4. Ruta `/salud`

Esta versión no solo dice que el proceso está vivo: **comprueba que la base de datos responde**. Si MariaDB cae, devuelve 503, y un `healthcheck` que use `/salud` marcará la API como `unhealthy`.

```
@app.route("/salud", methods=["GET"])
def salud():
    conexion = None
    cursor = None

    try:
        conexion = mysql.connector.connect(**DB_CONFIG)
        cursor = conexion.cursor()
        cursor.execute("SELECT 1")
        cursor.fetchone()
        return jsonify({"estado": "ok"}), 200

    except mysql.connector.Error:
        return jsonify({"estado": "error"}), 503

    finally:
```

```
        if cursor:
            cursor.close()
        if conexion and conexion.is_connected():
            conexion.close()
```

Línea	Qué hace
`@app.route("/salud", methods=["GET"])`	Decorador: "cuando llegue un GET a `/salud`, ejecuta la función de debajo".
`conexion = None` / `cursor = None`	Se crean vacías **antes** del `try`, para que el `finally` pueda mirarlas aunque la conexión falle a la primera.
`try:`	Todo lo que puede fallar va aquí dentro; si algo falla, salta al `except`.
`mysql.connector.connect(**DB_CONFIG)`	Intenta conectar a MariaDB con los datos de `DB_CONFIG` (host, usuario, contraseña…). Si la BD está parada o no resuelve `db`, lanza un error. Equivale a `conectar_db()`.
`conexion.cursor()`	El objeto que ejecuta SQL. Aquí sin `dictionary=True`: no hacen falta nombres de columna.
`cursor.execute("SELECT 1")`	La consulta **más barata posible**: no lee ninguna tabla, solo obliga al servidor a contestar. Si contesta, la conexión funciona de verdad.
`cursor.fetchone()`	Lee la respuesta (`(1,)`). Hay que leerla: con resultados pendientes, cerrar el cursor puede dar el error `Unread result found`.
`return jsonify({"estado": "ok"}), 200`	Todo bien: API **y** base de datos funcionan.
`except mysql.connector.Error:`	Cualquier error de BD: no conecta, contraseña mal, servidor caído.

Línea	Qué hace
`return jsonify({"estado": "error"}), 503`	**503 Service Unavailable** = "el servicio existe pero ahora no puede atender". No es 500 (fallo del programa) ni 404 (no existe).
`finally:`	Se ejecuta siempre, haya ido bien o mal (incluso después de un `return`).
`if cursor: cursor.close()`	Cierra el cursor solo si llegó a crearse.
`if conexion and conexion.is_connected(): conexion.close()`	Cierra la conexión solo si existe **y** sigue abierta, para no dejar conexiones colgadas en MariaDB (cada `/salud` abre una).

Cómo se prueba: `curl -i .../salud` → 200 · `docker compose stop db` → `curl` otra vez → 503 · `docker compose start db` → vuelve a 200 en unos segundos.

Ojo: con esta versión `/salud` ya **no** sirve para probar la imagen sin base de datos (daría 503). Si en el Dockerfile o en Compose pones un `HEALTHCHECK` contra `/salud`, la API saldrá `unhealthy` mientras `db` esté caída, que es justo lo que se quiere demostrar.

5. Ruta `/libros`

```
@app.route("/libros", methods=["GET"])
def obtener_libros():
    conexion = None
    cursor = None
    try:
        conexion = conectar_db()
        cursor = conexion.cursor(dictionary=True)
        cursor.execute("SELECT * FROM libros")
        libros = cursor.fetchall()
        return jsonify([normalizar_libro(l) for l in libros]), 200
    except mysql.connector.Error:
        app.logger.exception("Error consultando libros")
        return jsonify({"error": "Error en la base de datos"}), 500
    finally:
        if cursor:
            cursor.close()
        if conexion and conexion.is_connected():
            conexion.close()
```

Línea	Qué hace
`conexion = None` / `cursor = None`	Se inicializan antes del `try` para que el `finally` pueda comprobarlos aunque la conexión falle.
`conexion = conectar_db()`	Una conexión **por petición**: sdimple y sin estado compartido entre workers de gunicorn.
`conexion.cursor(dictionary=True)`	El cursor ejecuta SQL. Con `dictionary=True` cada fila llega como `{"id": 1, "titulo": ...}` en vez de una tupla `(1, ...)`: así el JSON lleva nombres de columna.
`cursor.execute("SELECT * FROM libros")`	Consulta fija, sin datos del usuario: no necesita parámetros.
`cursor.fetchall()`	Trae **todas** las filas como lista.
`return jsonify([...]), 200`	Lista JSON con todos los libros.
`except mysql.connector.Error:`	Cualquier error de BD (no conecta, permiso denegado, tabla no existe).

Línea	Qué hace
`app.logger.exception(...)`	Escribe el error completo en el log (lo ves con `docker compose logs api`).
`return ..., 500`	Al cliente solo le dice "error en la base de datos": **nunca** se le enseña el error interno.
`finally:`	Se ejecuta siempre, haya ido bien o mal: cierra cursor y conexión para no dejar conexiones abiertas en MariaDB.

6. Ruta `/libros/<id>`

```
@app.route("/libros/<int:id>", methods=["GET"])
def obtener_libro(id):
    ...
        cursor.execute("SELECT * FROM libros WHERE id = %s", (id,))
        libro = cursor.fetchone()
        if libro is None:
            return jsonify({"error": "Libro no encontrado", "id": id}), 404
        return jsonify(normalizar_libro(libro)), 200
    ...
```

El resto (`try`, `except`, `finally`) es igual que en `/libros`.

Línea	Qué hace
`"/libros/<int:id>"`	`<int:id>` captura lo que vaya tras `/libros/` y lo pasa como parámetro `id`. Con `int:`, Flask **solo acepta números**: `/libros/abc` da 404 sin llegar a la BD.
`"... WHERE id = %s", (id,)`	**Consulta parametrizada.** `%s` es un hueco; el conector envía el valor aparte y lo escapa. Nunca `f"... WHERE id = {id}"`: concatenar texto del usuario en el SQL permite **inyección SQL**.
`(id,)`	Los parámetros van en una **tupla**; la coma es obligatoria (`(id)` sin coma es solo un número entre paréntesis).
`cursor.fetchone()`	Trae **una** fila, o `None` si no hay ninguna.

Línea	Qué hace
`if libro is None: ... 404`	Libro inexistente → 404, que es lo que pide el ejercicio.
`return jsonify(...), 200`	El libro encontrado.

7. Arranque directo

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

Solo se ejecuta con `python app.py` (servidor de **desarrollo** de Flask, para probar). En el contenedor no se usa: el `CMD` lanza **gunicorn**, que importa `app:app` y salta este bloque. `0.0.0.0` es obligatorio dentro del contenedor: con `127.0.0.1`, Docker no podría hacerle llegar las peticiones.

Cómo se prueba cada parte: `curl -i .../salud` → 200 · `.../libros` → lista · `.../libros/1` → un libro · `.../libros/999` → 404 · `.../libros/abc` → 404 de Flask. Si `/libros` da 500, mira `docker compose logs api`: el motivo real está ahí.

Rutas importantes

Piensa en **qué tipo de cosa** buscas: configuración → `/etc` · logs → `/var/log` · tu script → `/usr/local/bin` · datos que sirves → `/srv` o `/var/www`. Descripción oficial: `man hier`.

Ruta	Contiene
`/etc`	**Configuración** de todo el sistema
`/etc/passwd` · `group` · `shadow`	Usuarios · grupos · contraseñas
`/etc/sudoers.d/`	Reglas de sudo (sin punto en el nombre)
`/etc/ssh/sshd_config.d/`	Configuración del servidor SSH (`*.conf`)
`/etc/systemd/system/`	Servicios del administrador (`.wants/` = los que arrancan solos)
`/etc/network/interfaces` · `/etc/NetworkManager/`	Red con ifupdown · con NetworkManager

Ruta	Contiene
`/etc/nginx/sites-available/` · `sites-enabled/`	Sitios web · los activos (enlaces)
`/etc/nftables.conf`	Firewall
`/etc/mysql/mariadb.conf.d/`	Configuración de MariaDB (`*.cnf`)
`/etc/apt/sources.list.d/`	Repositorios
`/usr/bin` · `/usr/sbin`	Programas · programas de administración
`/usr/local/bin`	Scripts propios del administrador
`/usr/lib/systemd/system/`	Servicios de los paquetes (no se tocan)
`/var/log/`	Logs
`/var/lib/mysql/`	Datos de MariaDB
`/var/backups/`	Copias de seguridad
`/var/www/`	Webs
`/home/usuario` · `/root`	Carpetas personales · la de root
`/srv`	Datos que sirve el sistema
`/tmp`	Temporales (sticky bit)
`/dev` · `/mnt` · `/media`	Dispositivos · montajes manuales · montajes automáticos

Casi todo el software sigue el **patrón** `.d`: un fichero principal que incluye una carpeta, y tu configuración va en un fichero propio dentro. Cada programa tiene sus reglas (extensión, y si gana la primera o la última opción): léelas en la línea `include` del principal.

Tus puntos débiles

Repásalos antes de cada práctica y antes de la prueba. Casi todos son de **precisión**, no de conceptos.

1. **Mirar el prompt** antes de escribir: ¿PC, VM, root o MariaDB?
2. **Leer el comando** antes de Intro: nombres y rutas iguales en todas partes (`user` / `users` , `web` / `servidorWeb`). Usa **Tab**.

3. **Contraseña antes que caducidad**, y comprobar con `passwd -S` que no hay **L**.
4. `-m` sin valor y sin `-d`; **`-G`** mayúscula para el grupo adicional.
5. En carpetas, **`770`**, no `660`: la `x` es poder entrar.
6. Reglas de sudo en **`/etc/sudoers.d/`**, con `(root)`, sin `NOPASSWD`, y **`chmod 0440`**.
7. **`su -`** con guion, y **`exit`** entre usuario y usuario. Para ser root, **`sudo -i`**.
8. **`ssh`** es el cliente; **`sshd`**, el servidor.
9. Las **claves SSH se generan en el cliente**.
10. **Validar no basta:** comprueba qué se aplica (`sshd -T`, `sudo -l`, `nft list ruleset`).
11. **Editar no aplica:** `daemon-reload`, `nft -f`, `reload`.
12. **Adaptar los ejemplos de internet** a tu máquina, y desconfiar de su orden.
13. En MariaDB, **el `;`**, y textos entre comillas.
14. **Usuarios de base de datos siempre con contraseña.**
15. **Comprobar cada paso** antes del siguiente, y que una prueba falla **por el motivo correcto**.
16. **Pegar una línea cada vez**: pegar bloques mezcla sesiones y usuarios.